Loose ends of a small RL project: Poisson bandits, TD step sizes, and one seed too few

Yaxin Zheng

November 2026

Abstract

At the end of a course project on reinforcement learning (a Poisson bandit and two small MDPs) we had three things we could not really explain: how close one can get to the Lai–Robbins bound on a tiny bandit, why TD(0) with step $1/n$ was so slow, and whether results obtained with a single random seed mean anything. This note goes back to each of them on problems where the exact answer is known. On the bandit, KL-UCB and Thompson sampling approach the Lai–Robbins constant only very slowly and from opposite sides, even after 10^6 pulls, while a UCB with a wrong variance that beats the bound on one instance gets stuck in up to 38% of runs on others. For TD(0), the slowness of $1/n$ is a pure bias effect: with steps c/n the error decays like $n^{-\min(c(1-\gamma),\,1/2)}$, which we see to two decimals, so $1/n$ forgets its initialisation like $n^{-0.2}$ when $\gamma = 0.8$. Finally, rerunning the report on 100 seeds, 7 of the 14 checkable sentences we wrote hold on (almost) every seed, 5 on 80 to 89% of them, and 2 were mostly an artefact of the seed we happened to use.

1 Introduction

This note started as a list of things we did not understand at the end of a course project. The project itself was small: three exercises from a master's course on reinforcement learning, each one a toy problem with a known answer. Exercise 2.4 is a three-armed bandit with Poisson rewards, exercise 3.1 is the "green robot" (a two-state MDP where a robot searches, waits or recharges its battery), and exercise 3.2 is the "student dilemma", a seven-state MDP where a student chooses between resting and working. We implemented explore-then-commit, ε_n-greedy, policy iteration, Monte Carlo, TD(0) and Q-learning on them and wrote a report.

Three things in that report kept bothering us after.

First, explore-then-commit sometimes got stuck on a bad arm and we only said that it is "not sub-linear". We never compared with algorithms that are supposed to be optimal, and we did not know how close one can actually get to the Lai–Robbins lower bound on such a small problem.

Second, the step size $1/n$ in TD(0) was much slower than a constant step in all our runs. The theory we learned says $1/n$ satisfies the Robbins–Monro conditions and converges, so we just wrote "too slow in practice" and moved on. That is not really an explanation.

Third, every number for exercises 3.1 and 3.2 came from a single random seed. We wrote this as a limitation in the conclusion, but we did not check if it mattered.

So this note goes back to these three points. None of the results are new in the sense of the literature, and we cite where they come from. What we tried to do is to check them carefully on problems where we know the exact answer, and to see how far the theory goes in explaining what we actually saw. All code is in the same repository as the project and every figure can be regenerated with one command.

2 The three problems

Poisson bandit. Three arms with rewards $\mathcal{P}(2)$, $\mathcal{P}(3)$ and $\mathcal{P}(4)$, so the gaps are $\Delta = (2, 1, 0)$. The pseudo-regret after n pulls is $R_n = \sum_k \Delta_k \mathbb{E}[N_n(k)]$, and we estimate it by averaging over independent runs. In Section 3 we also use two-armed instances $(\mu - 1, \mu)$ for $\mu \in \{2, 4, 16, 64\}$, where the gap stays equal to one but the variance (which for a Poisson law is the mean) grows.

Green robot. Two states, *high* and *low*. In high the robot can search (reward 5, stays high with probability 0.3) or wait (reward 1). In low it can search (reward 5, stays low with probability 0.6, otherwise breaks down, gets -10 and is put back in high), wait (reward 1) or recharge (reward 0, goes to high). With $\gamma = 0.8$ the optimal policy is "search when high, recharge when low", written s/re, with $v^* = (16.03, 12.82)$. In Section 4 we only do policy evaluation of s/re, and we change γ.

Student dilemma. Seven states, three of them absorbing (rewards -10, 100 and -1000 when you enter them), two actions. With $\gamma = 0.8$ the optimal policy is to work in states 1, 2, 3 and rest in state 4. The report evaluated a fixed policy called SWWS with TD(0) and then ran Q-learning with $\varepsilon = 0.1$ and $\alpha \in \{0.5, 0.1, 0.01\}$.

For all TD and Q-learning runs we keep the protocol of the report: values start at zero, episodes start in a uniformly random state, robot episodes last 50 steps, and the step size of a state (or state–action pair) depends on its own visit count n.

3 Poisson bandits and the Lai–Robbins constant

3.1 What the lower bound says here

For a bandit where every arm belongs to the same one-parameter family, Lai and Robbins [9] showed that any algorithm which is "uniformly good" (its regret is $o(n^a)$ for every $a > 0$ on every instance of the family) must satisfy

$$\liminf_{n\to\infty} \frac{R_n}{\log n} \;\geq\; C := \sum_{k:\,\Delta_k>0} \frac{\Delta_k}{\mathrm{KL}(\mu_k, \mu^*)}, \tag{1}$$

where for Poisson laws $\mathrm{KL}(p, q) = p \log(p/q) + q - p$. On the project instance $\mathrm{KL}(2, 4) \approx 0.614$ and $\mathrm{KL}(3, 4) \approx 0.137$, so $C = 2/0.614 + 1/0.137 \approx 3.26 + 7.30 = 10.56$. Most of the constant comes from arm 2, the one that is close to the best.

Two things in (1) are easy to forget, and both matter below. The bound is only asymptotic, so at a finite n an algorithm can sit under the line $C \log n$ without contradicting anything. And it only applies to uniformly good algorithms. An algorithm that is tuned for one instance can beat $C \log n$ on that instance, it just has to pay somewhere else.

3.2 Algorithms

We compare four index policies. All of them pull each arm once and then pull the arm with the largest index.

UCB with a fixed σ: index $\hat{\mu}_k + \sqrt{2\sigma^2 \log t / N_k}$. This is the usual sub-Gaussian UCB. The catch is that a Poisson law with mean μ has variance μ, so there is no single good σ. We use $\sigma = 1$, which is what one would write without thinking, and an "oracle" $\sigma = \sqrt{\mu^*}$, the standard deviation of the best arm (the largest of the instance).

KL-UCB [5, 2]: index $\max\{q \geq \hat{\mu}_k : N_k \,\mathrm{KL}(\hat{\mu}_k, q) \leq \log t\}$, computed by bisection. We drop the $\log\log t$ term, as most of implementations do.

Thompson sampling [7]: with Jeffreys prior the posterior of arm k is Gamma$(1/2 + S_k,\ N_k)$ (shape, rate), where S_k is the sum of its rewards. At each round we sample one value per arm and pull the largest.

KL-UCB and Thompson sampling with this prior are both known to reach the constant C asymptotically for one-dimensional exponential families [2, 7]. Fixed-σ UCB does not, its regret is about $\sum_k 2\sigma^2 \log n / \Delta_k$, which can be larger or smaller than $C \log n$ depending on σ.

All runs of an algorithm are simulated at the same time in numpy (one row per run), which is what makes horizons of 10^6 possible on a laptop.

3.3 The project instance, up to a million pulls

In the first follow-up (in French, in the `extensions` folder of the repository) we already ran the four algorithms together with ETC(10) and ε_n-greedy up to $n = 10^5$. The short version: ETC(10) and ε_n-greedy have a tiny median regret but 5 runs out of 200 stay stuck on arm 2 for a long time, so their mean regret is around 2 000; the index policies never get stuck. Here we only follow KL-UCB and Thompson sampling, but much further, to $n = 10^6$ with 100 runs, to see what really happens to $R_n / \log n$.

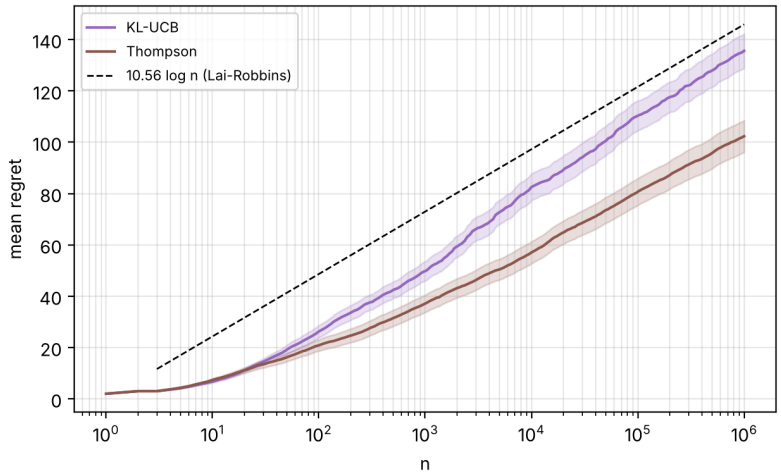


Figure 1: Mean regret of KL-UCB and Thompson sampling on the instance $(2, 3, 4)$, 100 runs, with ± 2 standard errors. The dashed line is $C \log n$.

At $n = 10^6$ the mean regret is 135.6 ± 3.3 for KL-UCB and 102.3 ± 3.1 for Thompson sampling (standard errors over runs). Divided by $\log n$ this gives 9.82 and 7.41, both *below* $C = 10.56$, which at first looks strange for algorithms that are supposed to be optimal and not better than optimal.

The ratio is the wrong thing to look at, though. If $R_n \approx a \log n + b$, then $R_n / \log n = a + b / \log n$, and b can be negative. A least squares fit of $R_n = a \log n + b$ between 10^4 and 10^6 gives $a = 11.9$, $b = -28$ for KL-UCB and $a = 9.8$, $b = -32$ for Thompson sampling. So both curves have a large negative offset, and the slope is what should go to C. The slope measured decade by decade tells the rest of the story:

KL-UCB comes down towards C from above, Thompson sampling goes up towards it from below, and on the last decade both are within about 10% of 10.56. With 100 runs these local slopes are not very precise (the standard error of the final regret is around 3), so we would not read the second decimal. But the picture is consistent with the theory, and it also says that even

decade of n	10^2–10^3	10^3–10^4	10^4–10^5	10^5–10^6
KL-UCB	9.95	14.32	12.41	11.35
Thompson sampling	7.13	8.39	10.03	9.45

Table 1: Local slope of the mean regret against $\log n$ on each decade (Lai–Robbins constant $C = 10.56$).

a million pulls on a three-armed problem is not really "asymptotic" yet. If we had stopped at $n = 10^4$, KL-UCB would have looked 35% worse than the bound in slope and Thompson sampling 20% better.

3.4 When the variance is wrong

On the instance $(2, 3, 4)$ of the French follow-up, UCB with $\sigma = 1$ had one of the best *median* regrets (32, only ETC(10) was a bit lower with 30), well below $C \log n$, but with one run out of 200 that stayed on the wrong arm for a long time. We wanted to know if this was luck or something structural. The bound says it cannot be free, so we looked for the instance where it pays.

We use two arms $(\mu - 1, \mu)$ with $\mu \in \{2, 4, 16, 64\}$. The gap is always 1, but the variance grows like μ and the constant grows with it: $C = 3.26$, 7.30, 31.3 and 127.3. For large μ the Poisson law is close to a Gaussian with variance μ and $C \approx 2\mu$, which is also the constant $2\sigma^2/\Delta$ of UCB with the oracle $\sigma = \sqrt{\mu}$. So on this family the oracle UCB should become almost optimal, and UCB with $\sigma = 1$ should become more and more over-confident. 100 runs of $n = 10^5$ pulls for each case.

	$\mu = 2$		$\mu = 4$		$\mu = 16$		$\mu = 64$	
	mean	median	mean	median	mean	median	mean	median
$C \log n$	37.5		84.1		360.7		1 466	
UCB ($\sigma = 1$)	25.2	24.5	585	21.0	19 495	19.5	38 066	23.5
runs stuck	0%		1%		20%		38%	
UCB (oracle σ)	43.7	42.0	87.8	84.0	321	300.5	1 124	1 074
KL-UCB	35.5	34.0	77.7	81.0	326	318.5	1 122	1 016.5
Thompson	26.3	27.0	55.2	50.5	182	160.0	657	654.5

Table 2: Regret at $n = 10^5$ on the two-armed family $(\mu - 1, \mu)$, 100 runs. "Stuck" means a final regret above $n/10$. The index policies other than UCB ($\sigma = 1$) never got stuck.

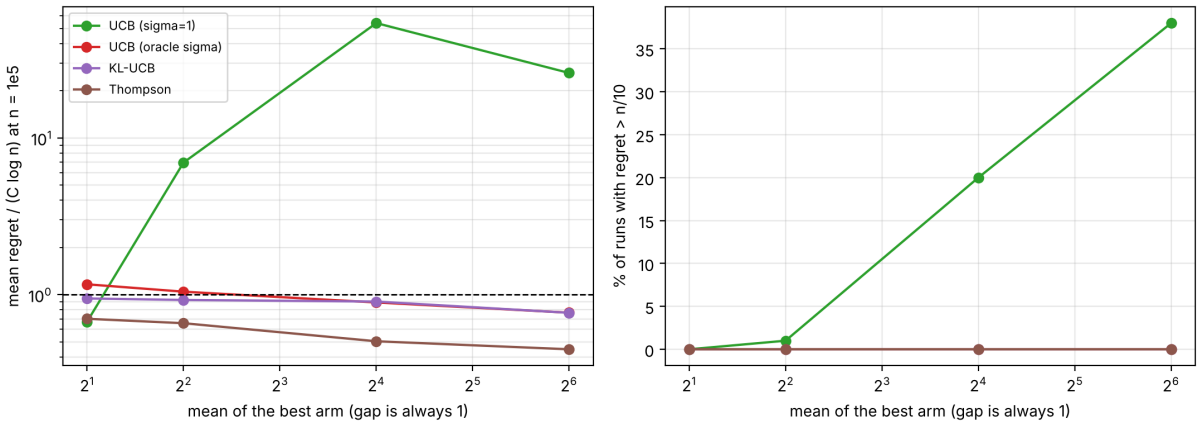


Figure 2: Left: mean regret divided by $C \log n$ at $n = 10^5$. Right: share of runs with regret above $n/10$.

Table 2 is quite brutal for UCB with $\sigma = 1$. Its median regret stays around 20 on every instance, which is far under $C \log n$ (about 18 times under it for $\mu = 16$ and 60 times for $\mu = 64$). If you only looked at medians, or at a few runs, you would think it is the best algorithm by far. But the share of runs stuck on the bad arm goes 0%, 1%, 20%, 38%, and the mean regret is 54 and 26 times the bound for the two largest instances. The confidence bonus $\sqrt{2 \log t / N}$ is simply too small when the real standard deviation is 4 or 8: after a few unlucky pulls of the best arm, its index can stay under the other one for a very long time. This is exactly the price the Lai–Robbins bound talks about. The algorithm is very good on some instances because it is not uniformly good.

The oracle UCB behaves as the Gaussian approximation predicts. On $\mu = 2$ it is clearly worse than KL-UCB (43.7 against 35.5), because there the Poisson law is far from Gaussian and $2\mu = 4$ is larger than $C = 3.26$. From $\mu = 16$ on it is as good as KL-UCB. Of course it needs to know the variance, while KL-UCB gets it for free from the shape of the KL.

Among the algorithms that never get stuck, Thompson sampling is the best everywhere and by a lot: at $n = 10^5$ its mean regret is between 0.45 and 0.70 times $C \log n$. Like for the long run on $(2, 3, 4)$, this is not a contradiction but a sign that we are still far from the asymptotic regime. The local slopes between 10^3 and 10^5 say the same thing. For KL-UCB they are all above C (1.3 to 1.6 times C), for Thompson sampling all below (0.6 to 0.85 times C). Both algorithms are supposed to end at the same constant, but they come from opposite sides, and on these problems the side of Thompson sampling is the comfortable one.

4 Why the step size $1/n$ is so slow

4.1 A one-state calculation

Take a single state that loops on itself with a deterministic reward r. Its value is $v = r/(1-\gamma)$. TD(0) with step α_n gives $V_n = V_{n-1} + \alpha_n(r + \gamma V_{n-1} - V_{n-1})$, so the error $e_n = v - V_n$ satisfies

$$e_n = \big(1 - (1-\gamma)\,\alpha_n\big)\, e_{n-1}. \tag{2}$$

With $\alpha_n = 1/n$ the product can be written with Gamma functions,

$$\frac{e_n}{e_0} = \prod_{k=1}^{n} \frac{k - 1 + \gamma}{k} = \frac{\Gamma(n+\gamma)}{\Gamma(\gamma)\,\Gamma(n+1)} \sim \frac{n^{-(1-\gamma)}}{\Gamma(\gamma)}.$$

For $\gamma = 0.8$ this is $n^{-0.2}$. After 10^5 visits there is still 8.6% of the initial error left. On the robot we start from $V = 0$ and $v(\text{high}) = 16.03$, which would leave an error around 1.4. In report we measured 1.28 with one seed, and the mean over 100 seeds (Section 5) is 1.33. So the slowness we saw is not noise at all, it is the bias of the initialisation that the step $1/n$ forgets too slowly.

The intuition is simple once it is written. With $\alpha_n = 1/n$, V_n is just the average of all the targets since the beginning, and the first targets were computed with a wrong V. They are never forgotten, only diluted, and each new target is itself built on the biased V (that is the factor γ). If $\gamma = 0$ the step $1/n$ would be perfect. The closer γ is to one, the worse it gets.

4.2 What stochastic approximation predicts

With noise the picture is the classical one for stochastic approximation with steps c/n [8]. Let λ be the smallest real part of an eigenvalue of the matrix of the mean dynamics. Then the error behaves like $n^{-c\lambda}$ when $c\lambda < 1/2$ (the bias wins) and like $n^{-1/2}$ when $c\lambda > 1/2$ (the noise wins, which is the best one can hope for). For TD(0) with per-state counts, a state visited a fraction π_s of the time gets steps $c/(\pi_s t)$ but is updated only a fraction π_s of the time, so the mean dynamics

in global time t is $\dot{V} = (c/t)(R + \gamma PV - V)$. The matrix is $c(I - \gamma P)$. For s/re the transition matrix has eigenvalues 1 and -0.7, so the smallest eigenvalue of $I - \gamma P$ is $1 - \gamma$ and the prediction is

$$e_n \approx n^{-\min(c(1-\gamma),\, 1/2)}. \tag{3}$$

So $1/n$ (that is $c = 1$) is in the bad regime as soon as $\gamma > 1/2$, and c has to be at least $1/(2(1-\gamma))$, half the effective horizon $H = 1/(1-\gamma)$, to get the $n^{-1/2}$ rate. This is the same message as the results of Szepesvári [10] and Even-Dar and Mansour [4] for Q-learning: with $1/n$ the time needed is exponential in H, while with polynomial steps $1/n^{\omega}$, $\omega \in (1/2, 1)$, it is polynomial.

For a polynomial step (2) gives $e_n \approx e_0 \exp\big(-(1-\gamma)\, n^{1-\omega}/(1-\omega)\big)$, which kills the bias very fast, but the noise left is of order $\sqrt{\alpha_n} = n^{-\omega/2}$. So ω should not be too small either. We test these three predictions in the next subsections: the exponent in (3) as a function of c, how the cost grows with H, and which ω is best.

We use $\alpha_n = c/(n+c-1)$ instead of c/n so that the first step is exactly one and $\alpha_n \le 1$ always. For $c = 1/(1-\gamma)$ the product in (2) telescopes, $\prod_{k \le n}(k+c-2)/(k+c-1) = (c-1)/(n+c-1)$, which is a nice sanity check for the code.

4.3 The exponent really is $\min(c(1-\gamma), 1/2)$

We ran TD(0) on the robot (policy s/re, $\gamma = 0.8$) for 10^6 updates with $\alpha_n = c/(n+c-1)$ and ten values of c, 30 seeds each. The error is $\max_s |V(s) - v(s)| / \max_s |v(s)|$ and we fit the slope of log(mean error) against $\log n$ between 10^4 and 10^6 updates.

c	0.5	1	1.5	2	2.5	3	4	5	10	20
predicted slope	-0.10	-0.20	-0.30	-0.40	-0.50	-0.50	-0.50	-0.50	-0.50	-0.50
measured slope	-0.100	-0.200	-0.301	-0.402	-0.506	-0.604	-0.518	-0.504	-0.555	-0.483
$10^3\times$ final error	180	52	15.2	4.7	1.39	0.48	0.30	0.34	0.29	0.44

Table 3: Slope of the error in log-log scale for $\alpha_n = c/(n+c-1)$, robot, $\gamma = 0.8$, 30 seeds.

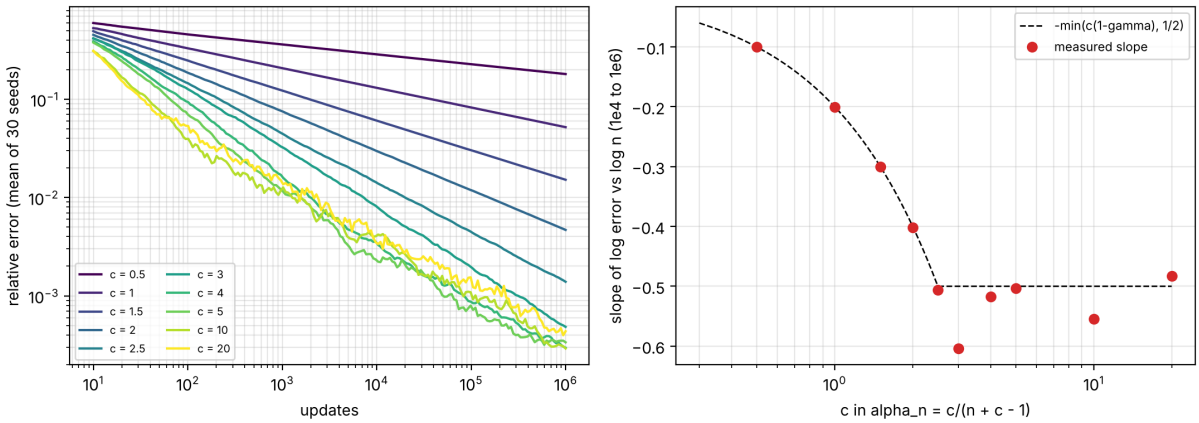


Figure 3: Left: relative error of TD(0) for different c. Right: measured slope against the prediction $-\min(c(1-\gamma), 1/2)$.

We did not expect the agreement to be this good. Up to $c = 2.5$ the slopes match the prediction to the second decimal (Table 3 and Figure 3). In this regime the error is almost deterministic, it is the bias of $V_0 = 0$ that decays, so in a way the one-state calculation is exactly what is measured. Above $c = 2.5$ we are in the noise regime and the slopes scatter around -0.5, between -0.48 and -0.60. The value -0.60 at $c = 3$ is because the bias term $n^{-0.6}$ is still larger than the noise in our window, it would go to -0.5 later.

So the step $1/n$ of the report is not "slow" in some vague sense. It sits at $c(1-\gamma) = 0.2$, deep in the regime where the initial error is forgotten at speed $n^{-0.2}$. Just taking $c = 4$ instead of $c = 1$ divides the final error by about 175. Very large c is a bit worse again ($c = 20$ gives 4.4×10^{-4}), because the noise constant grows with c.

4.4 How the cost grows with the horizon

Next we changed γ from 0.5 to 0.98, so the effective horizon $H = 1/(1-\gamma)$ goes from 2 to 50, and counted how many updates are needed before the mean relative error goes below 5% and stays there. The budget is 10^6 updates. Four rules are compared: $1/n$, $H/(n+H-1)$ (that is $c = H$), $1/n^{0.8}$ and $1/n^{0.6}$.

γ	0.5	0.7	0.8	0.9	0.95	0.98
H	2	3.3	5	10	20	50
$1/n$	135	6 905	–	–	–	–
one-state prediction	127	9.1×10^3	1.5×10^6	5×10^{12}	6×10^{25}	6×10^{64}
$H/(n+H-1)$	31	80	143	340	766	1 825
$1/n^{0.8}$	42	286	1 366	27 682	629 498	–
$1/n^{0.6}$	37	85	191	1 084	5 805	58 727

Table 4: Number of updates before the relative error stays under 5% (mean of 30 seeds). A dash means not reached within 10^6 updates.

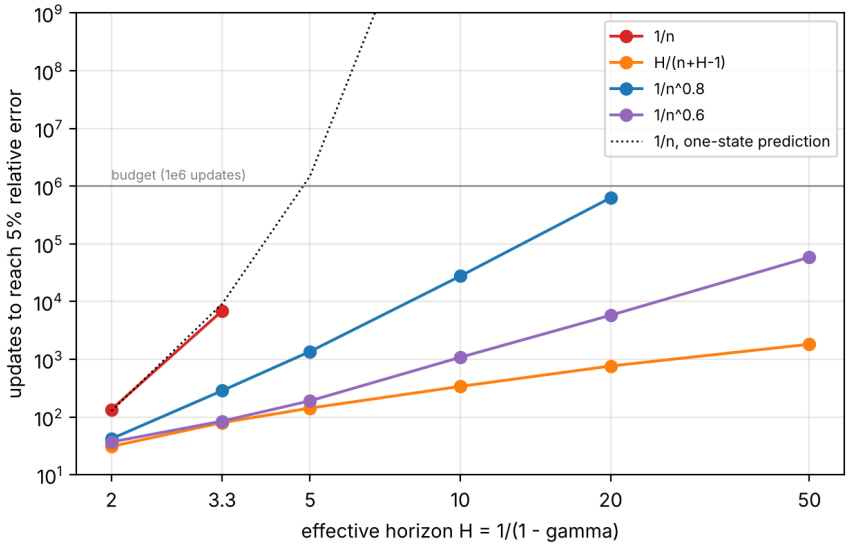


Figure 4: Updates needed to reach 5% relative error as a function of the horizon (log-log). The dotted line is the one-state prediction for $1/n$.

Table 4 is basically the Even-Dar and Mansour story on one small example. With $1/n$ the one-state formula $n \approx (0.05\,\Gamma(\gamma))^{-1/(1-\gamma)}$ predicts 127 updates for $\gamma = 0.5$ and about 9 100 for $\gamma = 0.7$, we measure 135 and 6 905. For $\gamma = 0.8$ it predicts 1.5×10^6, just above our budget, and indeed the error at 10^6 is 5.2%, so it misses by very little. Beyond that the predicted numbers are astronomical, 5×10^{12} updates for $\gamma = 0.9$, and of course nothing happens.

The rule $c = H$ is the opposite: the cost grows roughly linearly with the horizon, from 31 updates for $H = 2$ to 1 825 for $H = 50$ (a fit in log-log gives a power of 1.25). The polynomial steps are in between. They are polynomial in H as promised, but the power can be large. For $1/n^{0.8}$ the bias calculation gives $n \approx ((1-\omega)\,H \log 20)^{1/(1-\omega)}$, so roughly $(0.6\,H)^5$. That is 7.7×10^3 for

$H = 10$ and 2.5×10^5 for $H = 20$; we measure about three times more, which is the right order. With $\omega = 0.6$ the power is only 2.5 and the rule reaches the target even for $H = 50$.

4.5 Which ω?

If one does not want to depend on γ, a polynomial step $1/n^\omega$ is tempting. Figure 5 shows the relative error after 10^5 updates (averaged over the second half of the run and over 30 seeds) for ω from 0.5 to 1 and the same six values of γ.

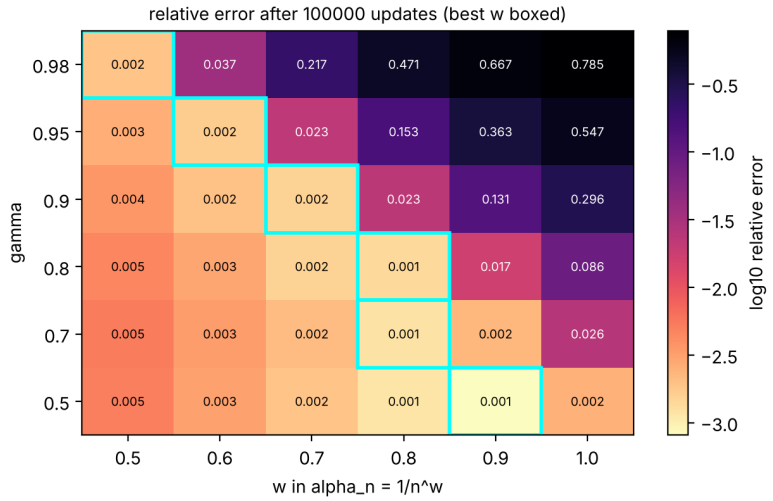


Figure 5: Relative error after 10^5 updates with $\alpha_n = 1/n^\omega$. The best ω for each γ is boxed.

The best ω goes down regularly when the horizon grows: 0.9 for $\gamma = 0.5$, 0.8 for 0.7 and 0.8, 0.7 for 0.9, 0.6 for 0.95 and 0.5 for 0.98. This is what the bias-variance argument says. A longer horizon means the bias is forgotten more slowly, so one needs larger steps (smaller ω) and accepts a bit more noise. The cost of a wrong ω is very asymmetric. In our grid, taking ω too small costs at most a factor of about 6, while taking it too large can cost more than two orders of magnitude (0.785 against 0.0019 for $\gamma = 0.98$).

One funny detail: $\omega = 0.5$ has the best worst case of the whole grid (at most 0.0053), but it is outside the Robbins–Monro conditions since $\sum_n 1/n$ diverges, so nothing guarantees it converges. For a fixed budget it does not matter much. If γ is known, the rule $c/(n + c - 1)$ with c around H is still the simplest choice, it reached 2 to 3×10^{-4} relative error after 10^6 updates for every γ we tried.

5 How much can one seed tell you?

Everything in the report for exercises 3.1 and 3.2 came from one seed per experiment (31 for the robot, 32 for the student). This is common in course projects, and to be honest also in a part of the literature [6]. Here we rerun the exact same experiments, same functions and same order of random draws, on 100 seeds (seeds 1 to 100, so the two seeds of the report are among them). We checked first that the seeds of the report give back the numbers of the report to the last digit, otherwise the comparison would not mean much.

We look at it from three angles. First we go through the report and pick the sentences that make a claim we can check on one run ("$\eta = 0.01$ gives the smallest error", "the three values of α end on the optimal policy", and so on), and count on how many of the 100 seeds the sentence would still be true. Second, for a few comparisons between two settings, we ask how many seeds

one has to average before the ranking agrees with the 100-seed ranking most of the time (Colas et al. [3] do a real power analysis for this kind of question, we just bootstrap from our 100 seeds), and we report the probability of improvement $P(X < Y)$ between one random run of each setting, as suggested by Agarwal et al. [1]. Third, we look at the shape of some distributions across seeds, because a mean $\pm$ standard deviation can be misleading when the distribution is skewed or has two modes.

5.1 Which sentences of the report survive

	claim (as written in the report, more or less)	true on	95% interval
R1	robot TD: $\eta = 0.01$ gives the smallest error for s/w	100	[0.96, 1.00]
R2†	robot TD: $\eta = 0.01$ gives the smallest error for s/s	63	[0.53, 0.72]
R3	robot TD: $\eta = 0.01$ gives the smallest error for s/re	100	[0.96, 1.00]
R4	robot TD: with $1/n$ the error for s/re is still above 1	100	[0.96, 1.00]
R5	robot TD: s/s is the hardest policy for every constant step	100	[0.96, 1.00]
R6	robot MC: all six true values are inside the 95% intervals	80	[0.71, 0.87]
R7	robot MC ε-greedy with the real return finds s/re	100	[0.96, 1.00]
R8	robot MC ε-greedy with the immediate reward finds s/w	100	[0.96, 1.00]
S1	student TD: $\alpha = 0.01$ gives the smallest error	88	[0.80, 0.93]
S2	student TD: $1/n$ is worse than 0.01	88	[0.80, 0.93]
S3	student Q-learning: the three α all end on W/W/W/R	88	[0.80, 0.93]
S4	student Q-learning: $\alpha = 0.5$ ends on W/W/W/R	89	[0.81, 0.94]
S5	student Q-learning: $\alpha = 0.01$ is stable after 2 500 to 4 500 episodes	97	[0.92, 0.99]
S6	student Q-learning: $Q(2, R)$ is at least 3 below q^* with $\alpha = 0.01$	7	[0.03, 0.14]

Table 5: Number of seeds (out of 100) on which each claim of the report holds, with Wilson intervals for the frequency. †R2 is not in the report itself, we wrote it in the French follow-up from a mean over 30 seeds.

Table 5 sorts itself quite naturally in three groups. Seven claims hold on (almost) every seed: R1, R3, R4, R5, R7, R8 and S5. These are the ones where the effect is large, for example $\eta = 0.01$ against the other steps for s/w and s/re, or the policy found by Monte Carlo control. Five claims are true most of the time but not always (between 80 and 89 seeds). And two are really fragile.

The first fragile one is R2. For the policy s/s, which has the noisy breakdowns, $\eta = 0.01$ is the best step on only 63 seeds, and on the 37 others it is the step $1/n$. We did not write this sentence in the report, we wrote it in the French follow-up because it is true on average over 30 seeds (here 0.55 against 0.80 over 100 seeds). Funny enough, the seed of the report is one of the 37 where $1/n$ wins (0.39 against 0.46), so the table of the report already contradicted it and we did not notice. On average the claim is right, seed by seed it is not far from a coin flip, and the probability that one random run of $\eta = 0.01$ beats one random run of $1/n$ is only 0.63.

The second one is S6. In the report we found $Q(2, R) = 10.3$ against $q^* = 15.29$ and explained it by the fact that this action is rarely tried. The explanation is fine, but over 100 seeds the mean is 14.5 ± 1.5 and only 7 seeds are 3 or more below q^*. Our seed 32 is part of the 100 (we used seeds 1 to 100) and it gives the lowest value of all of them. So the size of the effect we commented on was the worst case.

Two of the "mostly true" claims are worth a word because the reason is not just noise. For R6 (all true values inside the Monte Carlo intervals) we read the intervals at two decimals like in the report; strictly it is 0 out of 100 because of the truncation bias explained in the French follow-up. And 80 out of 100 is about what one should expect anyway: five of the six intervals are real 95% intervals (in our runs each one covers the true value between 92% and 99% of the time), and $0.95^5 \approx 0.77$. Looking at six intervals and finding them all fine is not a 95% event, it is a small multiple-testing problem that we did not think of when writing.

For S3 and S4 the explanation is even simpler. With $\alpha = 0.5$ the share of the last 5 000 episodes where the greedy policy is optimal is 0.882 ± 0.021 across seeds, so it is very stable. The policy just keeps switching, and is right about 88% of the time. "It ends on the optimal policy" is then one draw of a coin with $p \approx 0.88$, and 89 seeds out of 100 is exactly what that coin gives. A sentence about the final policy of a learner that never settles is a statement about one random sample.

5.2 How many seeds would have been enough

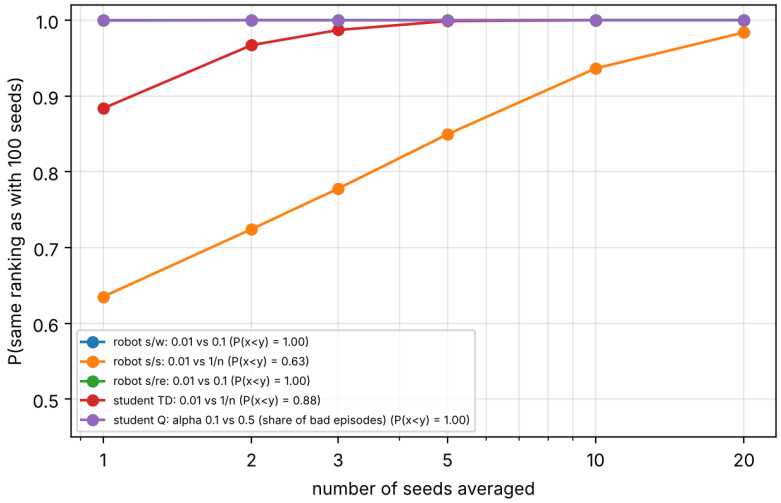


Figure 6: Probability that the mean over k random seeds ranks two settings like the mean over 100 seeds (bootstrap, 5 000 resamples). The three comparisons with $P(x < y) = 1$ are all at 1 and their lines are on top of each other.

For three of the five comparisons of Figure 6 one seed is enough: the probability of improvement is 1, every single seed ranks them in the same way. For the student, $\alpha = 0.01$ against $1/n$ has $P(X < Y) = 0.88$, and the ranking with one seed agrees with the 100-seed ranking 88% of the time, 97% with two seeds and 99% with three. The hard one is s/s on the robot, $\eta = 0.01$ against $1/n$, with $P(X < Y) = 0.63$: one seed agrees 64% of the time, five seeds 85%, ten seeds 94% and twenty seeds 98%.

Note that with one seed the agreement is just the probability of improvement (0.64 against 0.63, 0.88 against 0.88), which is obvious once you see it. A rough rule from our five cases: if $P(X < Y)$ is above 0.85, three seeds are enough; around 0.6 you need something like twenty. The problem is that you only know $P(X < Y)$ after running many seeds.

5.3 Means and standard deviations can hide a lot

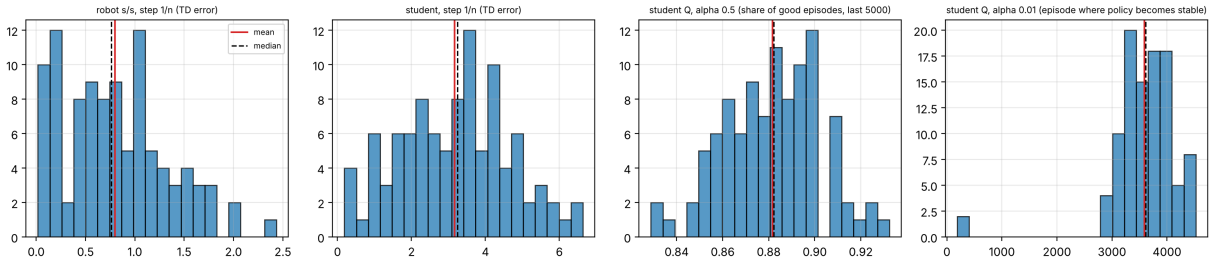


Figure 7: Distribution over 100 seeds of four quantities of the report. Red: mean, dashed: median.

Figure 7 shows four distributions over the 100 seeds. The TD error of $1/n$ on s/s goes from 0.02 to 2.44, with mean 0.80 and standard deviation 0.53. Written as 0.80 ± 0.53 it looks like a noisy but unimodal quantity, but in fact 23% of the seeds are under 0.3 and 35% are above 1. The same step on the student is wide but symmetric (3.17 ± 1.49, from 0.19 to 6.64), so there one seed can give anything between a very good and a very bad result. The episode where Q-learning with $\alpha = 0.01$ becomes stable is the opposite case: 98 seeds between 2 854 and 4 524 and two seeds at 190 and 249, where the exploration found the reward of state 6 early. Those two seeds pull the standard deviation up to 625, while the interquartile mean (3 628) and the median (3 615) are hardly affected. None of this is surprising for people who work on RL evaluation [1], but we would not have guessed it on problems this small.

6 What we take from this

Each of the three questions ended up with a cleaner answer than we expected when we started.

For the bandit, the Lai–Robbins constant is a statement about slopes, not about ratios. On a three-armed Poisson problem even a million pulls is not asymptotic: KL-UCB approaches C from above, Thompson sampling from below, and both have a large negative offset that makes $R_n / \log n$ look better than the bound. The other side of the bound is just as visible. UCB with $\sigma = 1$ has a beautiful median regret on every instance we tried, and it gets stuck in 38% of the runs when the means are around 64. KL-UCB needs no tuning at all, and Thompson sampling was the best of the algorithms that never got stuck.

For the step sizes, the slowness of $1/n$ is a bias problem, not a noise problem. The exponent $\min(c(1-\gamma), 1/2)$ from stochastic approximation is matched to two decimals, and it says that $1/n$ forgets the initial values like $n^{-0.2}$ when $\gamma = 0.8$. In practice we would now use $c/(n + c - 1)$ with c around the horizon $1/(1-\gamma)$, and if γ is not known, a polynomial step with ω around 0.6 (or even 0.5 for a fixed budget), keeping in mind that $\omega = 0.8$, which looks like a safe middle value, is bad for long horizons.

For the seeds, seven of the fourteen sentences we checked (thirteen from the report, one from the French follow-up) hold on every seed or almost, five hold on 80 to 89 seeds out of 100, and two were mostly produced by our particular seed. The fragile sentences are not random: they are the ones about small effects, about a final policy of a learner that never stops switching, or about the size of a bias that our seed happened to make extreme. If we rewrote the report today we would change exactly three things: drop the sentence on $Q(2, R)$ or give its value over many seeds, say that $\alpha = 0.5$ ends on the optimal policy about 88% of the time, and replace the step $1/n$ by $5/(n + 4)$ in the comparison of steps, since $1/n$ is not a fair representative of decreasing steps.

Limitations. Everything here is on three toy problems where the answer is known, which is exactly what made the checks possible but also limits how far the conclusions go. The step-size experiments are only on policy evaluation of one policy of the robot. We did not try Q-learning with the c/n rule beyond the quick test of the French follow-up, and we did not try iterate averaging (Polyak–Ruppert), which is the other classical way to get the $n^{-1/2}$ rate without knowing γ. For the bandits we used one family of Poisson instances and one horizon for the scale experiment. And 100 seeds is a lot for a course project but still gives intervals of a few percent on the claim frequencies.

What we would do next. The most interesting open question for us is the second one in a setting where γ is not known, or where the horizon changes from state to state. The heat map of Figure 5 suggests that a rule which adapts ω (or c) to the observed mixing time could work, and

it would be nice to see if this can be done with a simple estimate. On the bandit side, it would be interesting to measure on the scale family how many pulls it takes before the local slope is within 10% of C, as a function of μ.

Reproducibility. All scripts, raw results and figures are in the `research` folder of the repository. Each experiment runs in a few minutes on a laptop, the longest one (KL-UCB up to 10^6) in about ten.

References

[1] R. Agarwal, M. Schwarzer, P. S. Castro, A. Courville, M. G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. *NeurIPS*, 2021.

[2] O. Cappé, A. Garivier, O.-A. Maillard, R. Munos, G. Stoltz. Kullback–Leibler upper confidence bounds for optimal sequential allocation. *Annals of Statistics*, 41(3), 2013.

[3] C. Colas, O. Sigaud, P.-Y. Oudeyer. How many random seeds? Statistical power analysis in deep reinforcement learning experiments. *arXiv:1806.08295*, 2018.

[4] E. Even-Dar, Y. Mansour. Learning rates for Q-learning. *Journal of Machine Learning Research*, 5, 2003.

[5] A. Garivier, O. Cappé. The KL-UCB algorithm for bounded stochastic bandits and beyond. *COLT*, 2011.

[6] P. Henderson, R. Islam, P. Bachman, J. Pineau, D. Precup, D. Meger. Deep reinforcement learning that matters. *AAAI*, 2018.

[7] N. Korda, E. Kaufmann, R. Munos. Thompson sampling for 1-dimensional exponential family bandits. *NeurIPS*, 2013.

[8] H. J. Kushner, G. G. Yin. *Stochastic Approximation and Recursive Algorithms and Applications.* Springer, 2nd edition, 2003.

[9] T. L. Lai, H. Robbins. Asymptotically efficient adaptive allocation rules. *Advances in Applied Mathematics*, 6(1), 1985.

[10] C. Szepesvári. The asymptotic convergence-rate of Q-learning. *NeurIPS*, 1997.